Task 2 — Task/To-Do CRUD Management

Overview

Build a per-user to-do manager that supports the full create-read-update-delete lifecycle with strict ownership enforcement. This task exercises resource ownership, validation, and reactive list rendering.

User Story

> ***As a*** *logged-in user,* ***I want*** *to create, view, update, and delete my tasks,* ***so that*** *I can track my daily work in one place.*

Your Story Point Estimation (To Be Completed by Learner)

Estimate the effort for this user story using a Fibonacci scale (1, 2, 3, 5, 8, 13, ...). Consider complexity, effort, risk, and uncertainty, then justify your reasoning.

Factor	Your Assessment
Complexity *(how hard is the logic?)*	*(fill in)*
Effort *(how much work / how many parts?)*	*(fill in)*
Uncertainty / Risk *(unknowns, dependencies?)*	*(fill in)*
Your Estimate (points)	*(fill in)*
Rationale	*(explain how you arrived at your estimate)*

Acceptance Criteria

- A user can only see and modify **their own** tasks; access to others' tasks returns `403` / `404`.
- Each task has a title (required, non-empty), description (optional), status (`open` / `done`), and timestamps.
- Creating a task returns `201` with the created resource.
- Updating supports editing title/description and toggling status.
- Deleting a task removes it permanently and updates the list immediately.
- Requests referencing a missing task ID return `404`.

Data Model

Field	Type	Constraints
`id`	UUID / int	Primary key
`ownerId`	reference	Indexed; ties task to a user
`title`	string	Required, non-empty, trimmed
`description`	string	Optional
`status`	enum	`open` | `done` (default `open`)
`createdAt` / `updatedAt`	timestamp	Managed automatically

API Contract (Framework Agnostic)

Method	Endpoint	Purpose	Success	Errors
`POST`	`/tasks`	Create a task	`201`	`400`, `401`
`GET`	`/tasks`	List own tasks (optional `?status=`)	`200`	`401`
`PUT`	`/tasks/:id`	Update / toggle a task	`200`	`400`, `403/404`
`DELETE`	`/tasks/:id`	Delete a task	`204`	`403/404`

Back-End Requirements

- Enforce ownership on every read/write so users cannot access others' tasks.
- Validate required fields; reject empty/whitespace-only titles.

- Return `404` for missing IDs and `403` / `404` for cross-owner access.
- Support optional filtering by status on the list endpoint.

Front-End Requirements

- A task list with add, inline-edit, toggle-complete, and delete controls.
- Optimistic UI update (or refresh) after each operation, with rollback on error.
- Visual distinction between `open` and `done` tasks (e.g., strikethrough).
- Empty-state message when no tasks exist; optional filter tabs (All / Open / Done).

Non-Functional

- List rendering should remain responsive with dozens of tasks.
- All write actions provide feedback (success/error toast or inline message).

Edge Cases to Handle

- Creating a task with a whitespace-only title.
- Toggling status rapidly (double-click) without duplicating requests.
- Deleting a task that was already deleted in another tab.
- Attempting to edit/delete a task belonging to another user.

Unit Test Scenarios (Given / When / Then)

#	Given	When	Then
1	An empty/whitespace title	Create is called	Validation fails with `400`
2	A task owned by user A	User B updates/ deletes it	Access is denied (`403/404`)
3	An `open` task	Toggle is called	Status becomes `done` (and back)
4	A non-existent task ID	Delete is called	Response is `404` (not-found)
5	Several tasks with mixed status	List with `?status=open`	Only open tasks are returned

Deliverables

- Back-end CRUD endpoints with ownership enforcement and validation.
- Front-end list UI with add/edit/toggle/delete and empty state.
- Unit tests covering all scenarios above.
- Short README: how to run the app and tests, plus your completed estimation table.